

Boosting

Combines the predictions of several base models (often weak learners) to create a strong predictive model. The key idea behind boosting is to adjust the weights of observations based on the errors made by the previous models, allowing subsequent models to focus more on difficult cases or patterns.

Weak learners: is a model that performs slightly better than random guessing. Decision trees with very limited depth (stumps) are commonly used as weak learners in boosting.

Ensemble learning: Boosting combines multiple models to improve performance.

Sequential learning: Boosting trains the models sequentially. Each new model is trained to correct the errors made by the previous models.

Weighting Observations: After each iteration, the weights of misclassified instances are increased and the weights of correctly identified instances are decreased.

Stump: Decision Stump is a type of decision tree. It is a one-level decision tree that acts as the base classifier in many ensemble methods.

AdaBoost

1. Initialize the dataset and assign equal weights to each data point

$$w_i = \frac{1}{N} \quad , \quad N: \# \text{ training instances}$$

2. Train weak learners

For each iteration t :

2.1. Train a weak learner $h_t(x)$ on the weighted dataset.

2.2. Calculate the weighted error ϵ_t :

$$\epsilon_t = \frac{\sum_{i=1}^N w_i \cdot \mathbb{I}(y_i \neq h_t(x_i))}{\sum_{i=1}^N w_i}$$

↪ weighted missclassification rate
(classification Tasks)

$$\frac{\sum_{i=1}^N w_i \cdot (y_i - \hat{y}_i)^2}{\sum_{i=1}^N w_i}$$

↪ weighted MSE
(Regression Tasks)

$\mathbb{I}(y_i \neq h_t(x_i))$ is an indicator function that equals 1 if the i^{th} prediction by t^{th} model is incorrect

3. Calculate the classifier weight:

The weight α_t of the weak learner is computed as

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

$\epsilon_t \uparrow \quad \alpha_t$
 $0 \leq \epsilon_t \leq 1$

$$\frac{1}{2} \ln \left(\frac{1}{\epsilon_t} - 1 \right) \quad \epsilon_t \uparrow \quad \frac{1}{\epsilon_t} \downarrow \quad \ln \left(\frac{1}{\epsilon_t} \right) \downarrow \quad \alpha_t \downarrow$$

$$\epsilon_t = \underline{0.1} \rightarrow \alpha_t = 1.09$$

$$\epsilon_t = 0.9 \rightarrow \alpha_t = -1.09$$

$$\epsilon_t = 0.5 \rightarrow \alpha_t = 0$$

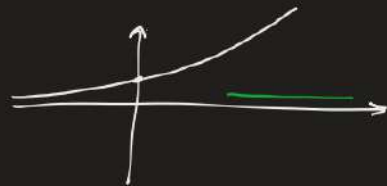
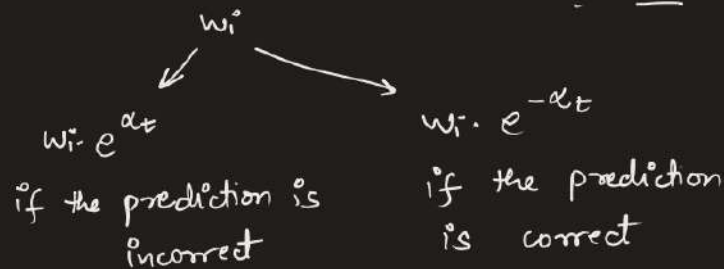
$\downarrow$
Random

4. update the weights of instances.

4.1 After the weak learner is trained, update the weights of the instances:

$$w_i \leftarrow w_i \cdot e^{-\alpha_t \cdot \underbrace{y_i \cdot h_t(x_i)}}_1$$

$y_i \cdot h_t(x_i)$ $\begin{cases} +1 & \text{if the prediction is correct} \\ -1 & \text{if the prediction is incorrect} \end{cases}$



> The weight of incorrect prediction will increase if $\alpha_t > 0$ General case

> The weight of correct prediction will increase if $\alpha_t < 0$

$\downarrow$
model is worse than
than the random
classifier

4.2 Normalize the weights so they sum up to 1:

$$w_i = \frac{w_i}{\sum_{j=1}^N w_j}$$

5. Repeat steps 2,3,4 until a stopping criteria is reached
(# iterations, $\epsilon_t < \text{threshold}, \dots$)

6. After training multiple weak learners and updating weights iteratively, the final prediction for a new instance is made by combining the predictions of all the weak learners weighted by their respective α_t :

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t \cdot h_t(x) \right)$$

T : # learners, $h_t(x)$: prediction of t^{th} learner

α_t : weight of t^{th} learner.

eg. #	Feature	True label (y)
1	1	1
2	2	1
3	3	-1

1. Initialize $w_1 = w_2 = w_3 = 1/3$

2. Train weak learner $h_1(x)$, let's assume the predictions are:

$$\begin{aligned}
 h_1(1) &= 1 \quad (\text{correct prediction}) & y_1 \cdot h_1(1) &= 1 \\
 \rightarrow h_1(2) &= -1 \quad (\text{incorrect prediction}) & y_2 \cdot h_1(2) &= -1 \\
 h_1(3) &= -1 \quad (\text{correct prediction}) & y_3 \cdot h_1(3) &= 1
 \end{aligned}$$

$$\epsilon_1 = \frac{1/3}{1/3 + 1/3 + 1/3} = 1/3 < \underline{0.5} \quad (\text{better than random})$$

$$3. \quad \alpha_1 = \frac{1}{2} \ln \left(\frac{1 - \epsilon_1}{\epsilon_1} \right) = \frac{1}{2} \ln \left(\frac{2/3}{1/3} \right) \approx 0.347$$

4. update weights of data points

$$\text{(correct)} \quad w_1 \leftarrow \frac{1}{3} \cdot e^{-0.347 \times 1} \approx 0.236$$

$$\text{(Incorrect)} \quad w_2 \leftarrow \frac{1}{3} e^{-0.347 \times -1} \approx 0.472$$

$$\text{(correct)} \quad w_3 \leftarrow \frac{1}{3} \cdot e^{-0.347} \approx 0.236$$

$$\text{Normalize:-} \quad w_1 = \frac{0.236}{0.236 + 0.472 + 0.236} \approx 0.25$$

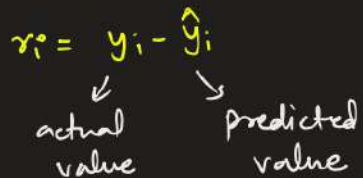
$$w_2 = \frac{0.472}{0.236 + 0.472 + 0.236} \approx 0.5$$

$$w_3 = \frac{0.236}{0.236 + 0.472 + 0.236} \approx 0.25$$

Gradient Boosting

1. Initialization: Start with an initial prediction, often the mean value of the target variable (for regression) or the log-odds of the target classes (for classification)
2. Calculate the residuals
 - > Compute the residuals (or errors) from the predictions of the current model

$$r_i = y_i - \hat{y}_i$$



3. Fit a new model

Train a new weak learner on the residuals. The new model tries to predict the residuals rather than the actual target values.

4. Update predictions.

Update the predictions by adding the new model's predictions, scaled by a learning rate α :

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \alpha \cdot h_t(x_i)$$

where $h_t(x_i)$ is the prediction of the new model.

5. Repeat steps 2-4 for a predefined number of steps or a threshold on error/residuals.
6. Final prediction:- weighted sum of all the weak learners

$$y(x) = h_1(x) + \alpha \cdot h_2(x) + \alpha \cdot h_3(x) + \dots$$

eg.

	Feature	y_i (actual)
1	1	10
2	2	15
3	3	20

$$\alpha = 0.9$$

1. Initialize : Baseline model :- predicts average value for all data points

$$\bar{y}_i = \frac{10+15+20}{3} = 15$$

$$\hat{y}_1^{(0)} = 15, \quad \hat{y}_2^{(1)} = 15, \quad \hat{y}_3^{(1)} = 15$$

2. Train a weak learner (regression) which learns the residuals

		$y_i - \hat{y}_i$	
	Feature	y_i	Residual
1	1	10	-5 ✓
2	2	15	0 ✓
3	3	20	5 ✓

3. Let's assume the weak learner gives the following predictions:

$$\underline{h_1(1)} = -7, \quad \underline{h_1(2)} = 0.8, \quad \underline{h_1(3)} = 6$$

$$4. \quad \hat{y}_1^{(2)} = 15 + (0.9) \times -7 = 8.7$$

$$\hat{y}_2^{(2)} = 15 + (0.9) \times 0.8 = 15.72$$

$$\hat{y}_3^{(2)} = 15 + (0.9) \times 6 = 20.4$$

5. Repeat

	Feature	y_i Actual	$\hat{y}_i$ Prediction	$y_i - \hat{y}_i$ Residual	
1	1	10	8.7	1.3	✓
2	2	15	15.72	-0.72	✓
3	3	20	20.4	-0.4	✓

...

XG Boost

XGBoost, which stands for Extreme Gradient Boosting, is an efficient and scalable implementation of the gradient boosting framework. It has gained immense popularity in the machine learning community due to its performance and speed. XGBoost is widely used in Kaggle competitions and in practical applications for classification and regression tasks.

Key Features of XGBoost

1. **Speed and Performance:** XGBoost is designed to be fast and efficient. It uses a more optimized implementation of gradient boosting, allowing for faster computation and better use of system resources.
2. **Handling Missing Values:** XGBoost can automatically handle missing values during training and prediction, making it robust in real-world scenarios.
3. **Regularization:** XGBoost includes L1 (Lasso) and L2 (Ridge) regularization techniques, which help reduce overfitting and improve model generalization.
4. **Tree Pruning:** It uses a more sophisticated approach to pruning trees. Instead of post-pruning, XGBoost employs a "max depth" parameter to control the complexity of the trees, which allows for better optimization.
5. **Parallelization:** XGBoost can build trees in parallel, leveraging multi-core processors for faster computation.
6. **Custom Objective Functions:** It allows users to define custom objective functions and evaluation criteria, providing flexibility for different types of tasks.

Light GBM & Cat Boost: These are advanced implementations of gradient boosting designed for speed and efficiency, particularly for large datasets.